



## BLOCKCHAIN INFRASTRUCTURE, PROFESSIONALLY OPERATED

**The infrastructure layer between institutional applications and blockchain networks.**

Dedicated access, protocol-specific operations and clear accountability—without building an internal blockchain infrastructure team.

ACCESS IS SIMPLE. OPERATIONS ARE NOT.

A blockchain endpoint hides a protocol-specific operating stack.

Institutions may only need dependable connectivity. Running it internally means owning an entirely different discipline.

|                                            |                                                  |
|--------------------------------------------|--------------------------------------------------|
| <div>01</div> <div>Network upgrades</div>  | <div>02</div> <div>State growth &amp; sync</div> |
| <div>03</div> <div>Peer connectivity</div> | <div>04</div> <div>Version compatibility</div>   |
| <div>05</div> <div>Hardware tuning</div>   | <div>06</div> <div>Monitoring &amp; alerts</div> |
| <div>07</div> <div>Incident response</div> | <div>08</div> <div>Protocol expertise</div>      |

The operational burden continues for as long as the application depends on the network.

BLOCKCHAIN ACCESS AS CRITICAL INFRASTRUCTURE

We absorb the infrastructure complexity. Your team consumes a dependable service.

YOUR ENVIRONMENT

Trading systems

Tokenization platforms

Custody & compliance

Data products



RUBY NODES OPERATING LAYER

Architecture

Monitoring

Performance

Deployment

Upgrades

Response



BLOCKCHAIN NETWORKS

L1 / L2

Appchains

## THE BUILD-VERSUS-OPERATE DECISION

**Keep protocol operations out of the critical path.**

Internal teams retain control of product architecture, keys, assets and business logic where appropriate. Ruby Nodes owns the agreed infrastructure scope.

### BUILD INTERNALLY



### MANAGED BY RUBY NODES

Recruit protocol expertise



**Use an experienced operating team**

Maintain every network



**One partner across networks**

Design monitoring & response



**Start with an operating baseline**

Absorb upgrade risk



**Managed lifecycle and upgrades**

Provision for uncertain load



**Capacity scoped to workload**

## WHAT WE OPERATE

Infrastructure matched to the workload.

01

### Managed access

Dedicated RPC, WebSocket and network-specific interfaces.

02

### Dedicated infrastructure

Isolated capacity instead of shared public endpoints.

03

### Archive & data

Historical state, indexing and high-throughput chain access.

04

### Network participation

Validators and specialized protocol roles where required.

## SPECIALIZED, HANDS-ON EXPERIENCE

**Operational credibility built in live networks.**

Our advantage is direct experience with infrastructure where configuration, performance and incident response have financial consequences.

**25+**

Validators operated

**100+**

Blockchain nodes operated

### Multi-protocol operations

Different software, failure modes and upgrade cycles

### Specialized network roles

Validator, committee, MPC and proving infrastructure

### Performance engineering

Benchmarking and tuning beyond default deployment

### Distributed operations

Infrastructure across multiple geographic regions

## WHERE THE MODEL APPLIES

Four workloads. One operating principle.

### 01 Trading firm

Need: Low-latency access across networks

**Ruby Nodes** — Dedicated nodes • optimized connectivity • performance tuning • managed upgrades

### 02 Bank / tokenization platform

Need: Dependable connectivity behind a financial product

**Ruby Nodes** — Production RPC • redundancy • archive data • lifecycle management

### 03 Analytics & compliance

Need: Reliable, high-volume chain data

**Ruby Nodes** — Archive nodes • high-throughput access • indexing • multi-network expansion

### 04 Investment firm / asset manager

Need: Participation without an internal operations team

**Ruby Nodes** — Validator infrastructure • monitoring • maintenance • defined key boundaries

## DEDICATED BY DESIGN

A clear infrastructure boundary.

### CLIENT ENVIRONMENT

Application

Trading / data systems

Keys & business logic



### MANAGED SERVICE BOUNDARY

Controlled access

Load balancing

Dedicated nodes

Monitoring

Redundancy

Incident response

Architecture varies by protocol, workload and required responsibility model.



### NETWORKS

## FROM REQUIREMENTS TO CONTINUOUS OPERATIONS

Ownership does not end at deployment.

01

### Discover

Networks, traffic, interfaces, regions and risk requirements.

02

### Design

Architecture, boundaries, resilience and service scope.

03

### Deploy

Provision, harden, synchronize and configure.

04

### Validate

Acceptance criteria for capacity, access and recovery.

## OPERATIONAL DISCIPLINE

**Controls appropriate to the service—not a generic checklist.**

The deployment model determines which controls, responsibilities and evidence apply. We make those decisions explicit before production.

---

### Access

Authorized access and minimum required exposure.

---

### Monitoring

Continuous service and infrastructure observability.

---

### Change

Protocol upgrades and configuration changes managed deliberately.

---

### Availability

Redundancy and failover where supported by the architecture.

---

### Response

Defined alerts, escalation paths and operational ownership.

---

### Custody

Asset control and signing responsibilities agreed separately.

---

Service levels, reporting and control evidence are defined for the contracted scope.

EXPERIENCE ACROSS PRODUCTION ECOSYSTEMS

Network breadth matters because protocols fail differently.

Operating across ecosystems builds practical knowledge of different clients, databases, consensus designs, hardware profiles and upgrade processes.

|     |        |          |          |
|-----|--------|----------|----------|
| Sui | Solana | Ethereum | Celestia |
| OG  | Walrus | zkVerify | peaq     |

## A PRACTICAL FIRST CONVERSATION

Tell us the networks, workloads and reliability requirements.

# We design and operate the infrastructure behind them.

### USEFUL INPUTS

Networks and required interfaces

Traffic and data-retention profile

Regions and latency requirements

Availability and responsibility model

---

**Petr Menšík**

[peter@rubynodes.io](mailto:peter@rubynodes.io)

[rubynodes.io](https://rubynodes.io)